A report on

Optimized Task Offloading for Healthcare IoT-Fog Systems

submitted in partial fulfillment of the requirements for

Summer Colloquium

by

Tanneeru Vidath: 2023BCS-068

Under the Supervision of

Dr. Chittaranjan Swain

Department of Computer Science and Engineering

ABV-INDIAN INSTITUTE OF INFORMATION TECHNOLOGY

AND MANAGEMENT GWALIOR

INDIA

MAY - AUGUST, 2026

DECLARATION

I hereby certify that the work, which is being presented in the report, entitled **Optimized Task Offloading for Healthcare IoT-Fog Systems**, submitted to the institution, is an authentic record of my own work carried out during the period May 2026 to July 2026 under the supervision of Dr. Chittaranjan Swain. I also cited the references about the text(s)/figure(s)/table(s) from where they have been taken.

Date: Signatures of the Candidate

This is to certify that the above statement made by the candidates is correct to the best of my knowledge.

Date: Signatures of the Supervisor

Acknowledgements

I would like to express my gratitude to Dr. Chittaranjan Swain for his guidance, patience, and advice throughout this project. His feedback helped shape the research direction and methodology.

I also thank the faculty of the Computer Science and Engineering Department for their encouragement and support.

Additionally, I appreciate the research facilities provided by ABV-IIITM Gwalior ,Finally, I thank my peers and colleagues for their feedback, discussions, and support during this work.

Tanneeru Vidath

(2023BCS-068)

Abstract

In healthcare Internet of Things (IoT) systems, computational tasks generated by medical devices must be processed by nearby Fog Nodes (FNs) within strict deadlines. The existing Multi-Stage Deferred Acceptance Based Fair Task Offloading (M-DAFTO) algorithm treats all tasks in a single undifferentiated pool, uses only two urgency criteria, and employs a trivially consistent 2×2 Analytic Hierarchy Process (AHP) matrix. Under high load, this causes critical surgical tasks to compete with routine checkups for the same resources, leading to unacceptable outages for life-critical operations.

This work adapts a severity-aware 2-Type Multi-Stage Deferred Acceptance (MSDA) algorithm that classifies tasks as Major (critical surgeries) or Minor (routine checkups) with separate quota constraints per fog node. The proposed approach extends the urgency function from two to four criteria by incorporating energy consumption and a severity multiplier, validated through a 4×4 AHP matrix with full consistency checking ($CR \leq 0.1$). Severity-weighted Min-Max normalisation is applied to both delay and energy, and separate preference lists are maintained for each task type. The 2-Type MSDA matching algorithm enforces three simultaneous constraints per fog node: Major quota limit, Minor quota limit, and total capacity limit.

The system was implemented in Java across 14 modular classes and evaluated against two baselines (Greedy Placement Method and M-DAFTO) over four task scales (250, 500, 1000, 2000) with 10 iterations each. The proposed method achieves 0% critical-task outage under normal load (250–1000 tasks) and reduces overall outage by approximately 79% and Major outage by approximately 89% on average compared to the baseline, while maintaining comparable task satisfaction and delay metrics.

Keywords: *Healthcare IoT, Fog Computing, Task Offloading, Matching Theory, 2-Type MSDA, Analytic Hierarchy Process (AHP), Resource Allocation.*

Contents

List of Acronyms

AHP	Analytic Hierarchy Process
CI	Consistency Index
CR	Consistency Ratio
DA	Deferred Acceptance
FN	Fog Node
GPM	Greedy Placement Method
IoT	Internet of Things
MCDM	Multi-Criteria Decision Making
MSDA	Multi-Stage Deferred Acceptance
PL	Preference List
RI	Random Index
VRU	Virtual Resource Unit

1 Introduction and Objectives

This chapter provides the context for this project and outlines the objectives to be achieved.

1.1 Context

The rapid growth of the Internet of Things (IoT) in healthcare has led to a proliferation of connected medical devices in smart hospital environments — patient monitors, surgical equipment, diagnostic imaging systems, and environmental sensors [1]. These devices generate computational tasks that must be processed in real time. For instance, a surgery monitoring system cannot tolerate the 100+ millisecond latency of a distant cloud data centre; it requires single-digit millisecond response times.

Fog computing addresses this need by placing small edge servers — called Fog Nodes (FNs) — close to the data source, typically within the same building or campus [2]. Compared to cloud computing, fog offers lower latency, reduced bandwidth consumption, higher reliability against internet outages, and stronger data privacy since sensitive patient information stays within the hospital network.

The core challenge addressed in this work is *task offloading*: deciding which FN should process which task. Each IoT task has a strict deadline, each FN has limited capacity measured in Virtual Resource Units (VRUs), and network bandwidth is shared. Critically, not all tasks are equally important — a major surgery matters far more than a routine checkup. If a task cannot be assigned to any FN, it results in an *outage*. In healthcare, outages for critical operations are unacceptable.

This problem is fundamentally a multi-criteria matching problem: tasks have preferences over FNs (based on delay and energy), FNs have preferences over tasks (based on urgency), and both sides have constraints (deadlines, quotas). Standard load balancing

optimises for a single metric and is insufficient. The Deferred Acceptance (DA) algorithm from matching theory [3] provides a principled framework, and Multi-Stage Deferred Acceptance (MSDA) extends it with minimum-quota enforcement for fairness [4].

1.2 Objectives

The primary base paper, M-DAFTO [1], proposes a Single-Type MSDA algorithm for IoT-fog task offloading. While effective, six key limitations can be identified:

- **Single-pool matching**: All tasks compete in one undifferentiated pool. A routine checkup competes for the same FN slots as a critical surgery, and under high load, critical tasks can be starved of resources.
- **2-criteria urgency**: The urgency function uses only delay difference and preference count. Energy consumption — a key concern in battery-powered IoT devices — is completely ignored.
- **No severity awareness**: There is no concept of task severity (Major vs Minor). The algorithm cannot prioritise critical healthcare operations over routine ones.
- **Trivially consistent 2×2 AHP**: With only 2 criteria, the Analytic Hierarchy Process (AHP) matrix is 2×2 and always passes the consistency check (CR = 0 by definition).
- **Delay-only normalisation**: Only delay is normalised via Min-Max scaling. Energy is neither normalised nor combined into the preference ordering.
- **Single preference list per FN**: Each FN has one preference ranking over all tasks. There are no separate rankings for different task types.

The main objective is to adapt and implement a severity-aware 2-Type MSDA algorithm from [5] that overcomes all six limitations, while maintaining the theoretical guarantees of the original MSDA framework. Specifically, the project aims to:

(i) Apply Major/Minor task classification with separate quota constraints based on the 2-Type framework of [5].

(ii) Extend the urgency function from 2 to 4 criteria (adding energy and severity).

(iii) Implement a 4×4 AHP matrix with genuine consistency validation.

(iv) Apply severity-weighted normalisation to both delay and energy.

(v) Generate separate fog-side preference lists for Major and Minor tasks, as required by the 2-Type MSDA architecture [5].

(vi) Evaluate the proposed method against two baselines across multiple load scales.

2 Literature Review

This chapter surveys the foundational theory and prior work relevant to this project, and identifies the gaps that motivate the proposed approach.

2.1 Matching Theory Foundations

The algorithm of Deferred Acceptance (DA), developed by Gale and Shapley [3] , is utilized for solving the stable matching problem for two-sided markets. A group of people, also called proposers, makes offers to another group called receivers. The receiver evaluates the proposals and tentatively accepts proposals that they consider to be the best ones, in terms of their capacity. Then rejected proposers come up with a new proposal to their next option. The process continues till no more proposals are available anymore.

Roth and Sotomayor [6] provided the foundational theoretical treatment of two side matching, establishing key properties such as strategy proofness for the proposing side and the lattice structure of stable matching. Their work forms the basis for applying matching theory to resource allocations problems beyond the original marriage and college admissions settings.

Standard DA does not enforce minimum quotas — some receivers may remain empty while others are overloaded. Kamada and Kojima [7] studied efficient matching under distributional constraints, showing that minimum-quota requirements can be incorporated while preserving desirable matching properties. Fragiadakis et al. [4] formalised the Multi-Stage Deferred Acceptance (MSDA) algorithm with minimum quotas. MSDA runs DA in multiple stages: in early stages, tasks with the latest deadlines are removed from the pool and DA runs with maximum quotas; in the final stage, minimum quotas force remaining tasks onto receivers even if constraints are relaxed. This ensures every receiver gets at least its minimum quota.

The extension of MSDA to handle two types of agents with separate quota constraints was proposed by Oomori and Manabe [5]. Their 2-Type MSDA maintains four quota values per receiver: minimum and maximum quotas for all agents, and separate minimum and maximum quotas for one designated type (R-type). This theoretical framework is the basis for the matching algorithm adapted in this project.

2.2 Task Offloading in IoT-Fog Systems

Chiti et al. [2] proposed a Matching-Based Task Offloading (MBTO) framework for IoT-fog systems, demonstrating that matching theory provides a principled approach to the task offloading problem. Their work showed that matching-based offloading outperforms greedy and random assignment strategies in terms of delay and resource utilisation.

Swain et al. [8] proposed LETO, a load-balanced task offloading strategy for IoT-fog systems that focused on distributing tasks fairly across fog nodes. LETO served as a direct predecessor to the M-DAFTO algorithm.

The M-DAFTO algorithm [1] extended the MSDA framework specifically for IoT-fog task offloading. M-DAFTO introduced a 2-criteria urgency function (based on delay difference and preference count), AHP-based weight generation, and multi-stage quota enforcement. M-DAFTO demonstrated improved fairness over prior approaches through its minimum-quota mechanism, which ensures that no fog node is left idle while others are overloaded.

2.3 Gaps Identified

Despite M-DAFTO's contributions, the following key gaps exist that limit its applicability to healthcare IoT:

(i) **No task differentiation**: M-DAFTO treats all tasks in a single pool. In healthcare, a critical surgery and a routine checkup should not compete equally for resources.

(ii) **Limited urgency criteria**: Using only delay difference and preference count ignores energy consumption, which is important for battery-powered IoT devices.

(iii) **No severity awareness**: Without a severity concept, the algorithm cannot guarantee that critical tasks are assigned even under high load.

(iv) **Trivial AHP consistency**: The 2×2 AHP matrix always has CR = 0, so no actual consistency validation occurs.

(v) **Incomplete normalisation**: Only delay is normalised; energy values are neither scaled nor incorporated into the preference ordering.

(vi) **Single preference ranking**: Each fog node maintains only one preference list over all tasks, preventing type-specific prioritisation.

These gaps motivate the adaptation of the 2-Type MSDA algorithm from [5], combined with the proposed extensions: expanded urgency criteria, genuine AHP consistency checking, severity-weighted normalisation, and the mapping of Major/Minor healthcare task severity onto the R-type/S-type framework.

3 System Model and Problem Formulation

This chapter describes the system model, formulates the task offloading problem mathematically, and provides a recap of the baseline M-DAFTO algorithm.

3.1 System Model

This work considers a healthcare IoT-fog system consisting of IoT devices and Fog Nodes (FNs), building upon the architecture defined in the M-DAFTO base model [1]. The IoT devices generate computational tasks, and the FNs process them. Figure 3.1 illustrates this architecture.

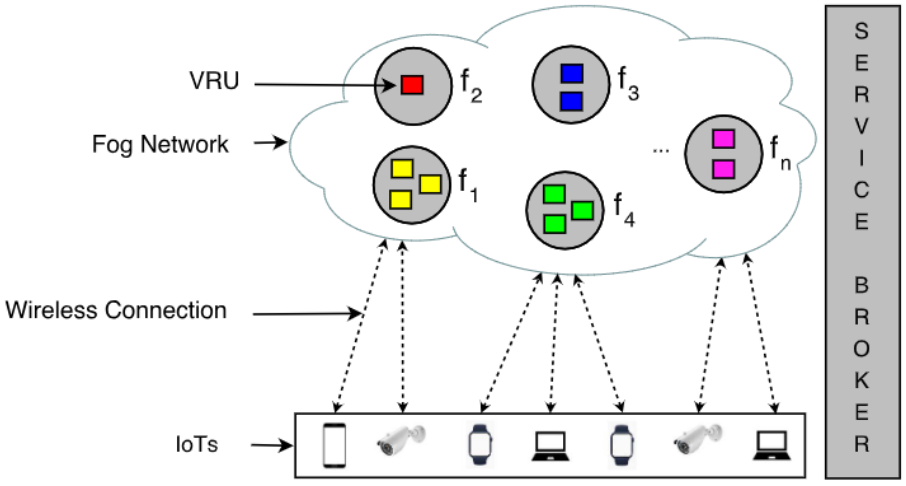

Figure 3.1: An IoT-Fog network comprising multiple IoT devices and FNs communicating over a wireless channel.

3.1.1 IoT Tasks

Each IoT task T_i (for $i = 1, 2, \ldots, M$) is characterised by:

- CpuDemanded$_i$: CPU demand in million cycles, drawn from $[210.0, 480.0]$ Mcycles.

- InputSize_i: Input data size in KB, drawn from $[300.0, 600.0]$ KB.
- OutputSize_i: Output data size in KB, drawn from $[10.0, 20.0]$ KB.
- Deadline_i: Processing deadline in seconds, drawn from $[15.0, 22.0]$ s.
- Phase_i: Either "Pre-Surgery" or "Post-Surgery" (assigned randomly with equal probability).
- Severity_i: Either "Major" (critical operations) or "Minor" (routine tasks), assigned randomly with equal probability.

3.1.2 Fog Nodes

Each FN F_j (for $j = 1, 2, \ldots, N$, where $N = 5$) is characterised by:

- totalCpuGHz_j: Total CPU capacity in GHz, drawn from $[6.0, 10.0]$ GHz.
- numberOfVRUs_j: Number of Virtual Resource Units (VRUs), drawn from $[200, 500]$.
- $\text{vruCapacity}_j = (\text{totalCpuGHz}_j \times 1000)/\text{numberOfVRUs}_j$ (Mcycles/s per VRU).

A VRU is a logical unit of compute capacity. A fog node with 400 VRUs can theoretically handle 400 concurrent tasks. The maximum quota of a fog node equals its number of VRUs.

3.2 Problem Formulation

3.2.1 Offloading Delay

For task T_i offloaded to fog node F_j, the offloading delay is:

$$\text{delay}_{ij} = \text{txTime}_{ij} + \text{execTime}_{ij} + \text{rxTime}_{ij} \tag{3.1}$$

where:

$$\text{txTime}_{ij} = \frac{\text{inputSize}_i}{\text{UPLINK\_DATA\_RATE}} \tag{3.2}$$

$$\text{execTime}_{ij} = \frac{\text{cpuDemanded}_i}{\text{vruCapacity}_j} \tag{3.3}$$

$$\text{rxTime}_{ij} = \frac{\text{outputSize}_i}{\text{DOWNLINK\_DATA\_RATE}} \tag{3.4}$$

The uplink and downlink data rates are both 2500.0 KB/s (derived from 20 MHz bandwidth).

3.2.2 Energy Consumption

The energy consumed when task T_i is offloaded to fog node F_j is:

$$\text{energy}_{ij} = (P_{\text{tx}} \times \text{txTime}_{ij}) + (P_{\text{exec}} \times \text{execTime}_{ij}) + (P_{\text{rx}} \times \text{rxTime}_{ij}) \tag{3.5}$$

where $P_{\text{tx}} = 0.5$ W (transmission power), $P_{\text{exec}} = 1.0$ W (execution power), and $P_{\text{rx}} = 0.5$ W (receiving power).

3.2.3 Assignment Constraints

A valid task assignment must satisfy:

- Each task is assigned to at most one fog node.
- The number of tasks assigned to fog node F_j must not exceed its maximum quota $q_j = \text{numberOfVRUs}_j$.
- The number of tasks assigned to F_j must meet its minimum quota p_j (for fairness).
- An unassigned task constitutes an *outage*.

3.3 Baseline M-DAFTO Recap

The M-DAFTO algorithm [1] uses a Single-Type MSDA (Algorithm 9 from [4]) with the following characteristics:

3.3.1 2-Criteria Urgency Function

For task T_i at fog node F_j:

$$U(T_i, F_j) = w_1 \times \frac{1}{\text{deadline}_i - \text{delay}_{ij}} + w_2 \times \frac{1}{\text{pref}(T_i)} \tag{3.6}$$

where $\text{pref}(T_i)$ is the number of fog nodes where $\text{delay}_{ij} \leq \text{deadline}_i$ (if 0, set to 1.0 to avoid division by zero), and w_1, w_2 are weights generated by a 2×2 AHP matrix.

3.3.2 2×2 AHP Weight Generation

M-DAFTO generates weights using a 2×2 pairwise comparison matrix. With only 2 criteria, the matrix always passes the consistency check (CR = 0 by definition), providing no actual validation.

3.3.3 Quota Determination

Minimum quotas are computed proportionally based on VRU capacity relative to the most efficient node. For each fog node F_j:

$$p_j = \min\left(\left\lfloor \frac{\text{vruCapacity}_j}{\lambda_{\max}} \times l_{\max} \right\rfloor,\ \text{numberOfVRUs}_j\right) \tag{3.7}$$

where $\lambda_{\max}$ is the maximum VRU capacity across all fog nodes, and $l_{\max} = \min(h_{\max}, \lfloor |T|/|F| \rfloor)$ with $h_{\max}$ being the number of VRUs of the most efficient node. Maximum quotas equal the number of VRUs.

3.3.4 Time Complexity

M-DAFTO achieves a time complexity of $O(m \cdot n)$ per DA round, where m is the number of tasks and n is the number of fog nodes [1].

4 Proposed Methodology

This chapter presents the proposed methodology in detail. The system architecture involves a complete pipeline from task generation and severity classification, through urgency scoring using an extended Analytic Hierarchy Process (AHP), to the final task assignment using an adapted 2-Type Multi-Stage Deferred Acceptance (MSDA) algorithm.

4.1 Overview of the Proposed Pipeline

The proposed system replaces the baseline M-DAFTO single-pool approach with a severity-aware matching pipeline. The pipeline operates in sequential stages to ensure that critical healthcare tasks are prioritised without entirely starving routine tasks. First, tasks are generated and classified by severity. Second, delay and energy costs are calculated and normalised. Third, a 4-criteria AHP model generates dynamic weights to compute an urgency score for each task-node pair. Finally, the adapted 2-Type MSDA algorithm uses these urgency scores to form preference lists and execute the matching process.

4.2 Severity-Aware Task Classification and Quotas

Unlike the baseline M-DAFTO algorithm which treats all tasks equally, the proposed system categorises tasks into two distinct classes based on the framework introduced by Oomori and Manabe [5]:

- **Major Tasks (R-type):** Critical healthcare operations, such as surgical robotic control or emergency patient monitoring.
- **Minor Tasks (S-type):** Routine healthcare operations, such as regular data logging or non-urgent checkups.

To ensure Major tasks are never starved of computational resources during high-load scenarios, the system enforces a strict quota structure on every fog node. Figure 4.1 illustrates this quota tracking mechanism. Each fog node maintains four quota values:

- p_c: minimum quota for all tasks
- q_c: maximum quota for all tasks (= numberOfVRUs)
- pr_c: minimum guaranteed quota for Major tasks
- qr_c: maximum allowed quota for Major tasks

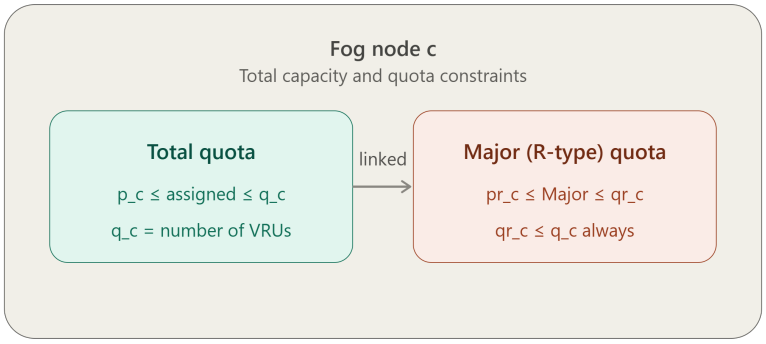

Figure 4.1: Fog node quota structure showing separate quota tracking for Major (R-type) and Minor (S-type) tasks, adapted from [5].

4.3 The Severity-Aware 4-Criteria Urgency Model

Prior to matching jobs with fog nodes, the first step is to evaluate and rank fog nodes in terms of urgency. The original M-DAFTO model is based on a simple two criteria urgency function (delay and preference number). In contrast the proposed system present a new four criterias model that include energy consumption and defined severity weighting.

4.3.1 Severity-Weighted Normalisation

Delay (measured in seconds) and Energy (measured in Joules) operate on vastly different scales. A severity-weighted Min-Max normalisation is introduced to align these

metrics:

$$\text{weight} = \begin{cases} 0.5 & \text{if severity} = \text{“Minor”} \\ 1.0 & \text{if severity} = \text{“Major”} \end{cases} \tag{4.1}$$

$$nd_{ij} = \frac{\text{delay}_{ij} - \text{minDelay}}{\text{maxDelay} - \text{minDelay}} \times \text{weight} \tag{4.2}$$

$$ne_{ij} = \frac{\text{energy}_{ij} - \text{minEnergy}}{\text{maxEnergy} - \text{minEnergy}} \times \text{weight} \tag{4.3}$$

4.3.2 Severity Score

To actively push major tasks higher in the preference ranking, a severity multiplier $\gamma(T_i)$ was dynamically calculated:

$$\gamma(T_i) = 1 + c_i \times 0.5 \tag{4.4}$$

where $c_i = 1$ for Major tasks and $c_i = 0$ for minor tasks. That force $\gamma = 1.5$ for critical tasks and $\gamma = 1.0$ for routine task.

4.3.3 4-Criteria Urgency Function and AHP

The final urgency function $U(T_i, F_j)$ for a task T_i on fog node F_j is computed as:

$$U(T_i, F_j) = w_1 \frac{1}{\Delta d} + w_2 \frac{1}{\text{pref}(T_i)} + w_3 \frac{1}{\text{energy}_{ij}} + w_4 \gamma(T_i) \tag{4.5}$$

where $\Delta d = \text{deadline}_i - \text{delay}_{ij}$.

The weights $w_1 \dots w_4$ are generated using a 4×4 Analytic Hierarchy Process (AHP) matrix. The weight generation process follows three rigorous steps [9]:

1. Pairwise Matrix Construction (PMC): Four random values $v[1..4]$ was drawn from $[1.0, 9.0]$. The pairwise comparisons matrix P is built where $P_{xy} = \text{getSaatyScale}(v_x/v_y)$ for $x < y$, with $P_{yx} = 1/P_{xy}$ and $P_{xx} = 1.0$.

2. Column Weight Determination (CWD): The matrix P is column normalised to produce $\bar{P}$, and the rows of $\bar{P}$ are averaged to obtain the final weights $W = [w_1, w_2, w_3, w_4]^T$.

3. Consistency Check (CC): Unlike the baseline's 2×2 matrix which is trivially

consistent (CR = 0 always), this 4×4 matrix requires genuine consistency validation. The maximum eigenvalue $\lambda_{\max}$ is computed:

$$\lambda_{\max} = \text{average}\left(\frac{\text{rowSum}(P \cdot W)}{W_x}\right) \tag{4.6}$$

$$CI = \frac{\lambda_{\max} - n}{n-1}, \quad RI = 0.90 \text{ (for } n = 4) \tag{4.7}$$

$$CR = \frac{CI}{RI} \tag{4.8}$$

The generated weights are only accepted if $CR \leq 0.1$. Otherwise, the matrix is regenerated.

4.4 The 2-Type MSDA Matching Engine

Before executing the matching algorithm, the system must generate preference and precedence lists to dictate the order of matching, as structurally required by the MSDA framework.

4.4.1 Preference and Precedence Lists

- **Task Preference Lists:** During the deferred acceptance process, each task T_i must propose to fog nodes in its order of preference. A task's preference list (`preferredFogIndices`) is constructed by sorting all available fog nodes in ascending order of their combined normalised cost, `normSum` ($nd_{ij} + ne_{ij}$). This ensures that tasks always propose to the most delay-and-energy efficient fog nodes first.

- **Fog Node Preference Lists:** Each fog node F_j evaluates all tasks using the 4-criteria urgency function $U(T_i, F_j)$ to determine its own preferences. It generates three separate sorted rankings: one for all tasks, one strictly for Major tasks, and one strictly for Minor tasks.

- **Task Precedence Lists:** To determine which tasks propose first during the deferred acceptance rounds, tasks are placed into precedence lists sorted strictly by

their tightest deadlines. Separate precedence lists are maintained for Major and Minor tasks.

These lists are then fed into the adapted 2-Type MSDA algorithm.

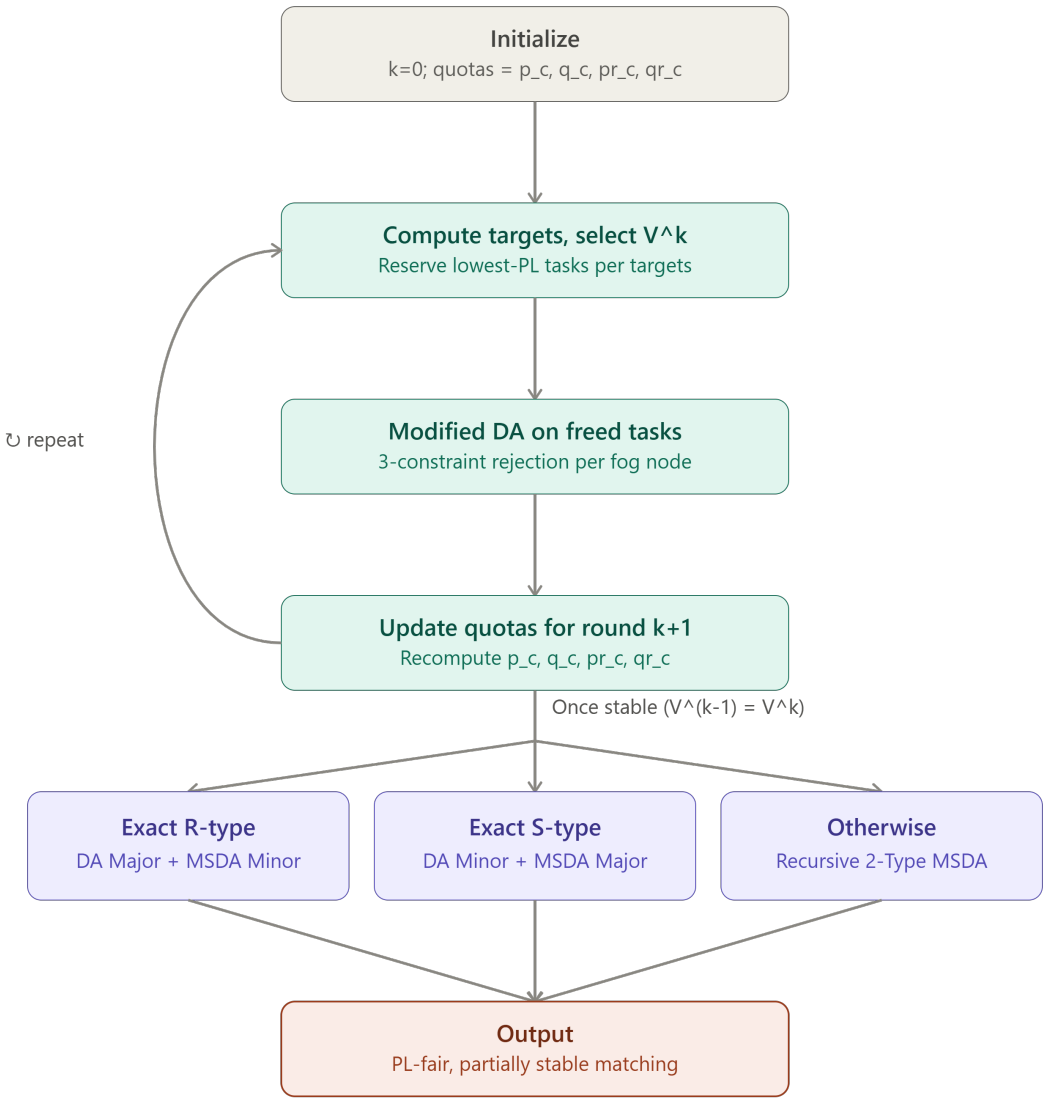

Figure 4.2: Flowchart of the adapted 2-Type MSDA matching engine, adapted from [5].

The matching engine utilises a Modified Deferred Acceptance (Modified DA) subroutine. As shown in Algorithm 1, this routine simultaneously enforces the strict Major and Minor capacity constraints defined by the fog node quotas.

Algorithm 1 Modified Deferred Acceptance (Fog Node Receiver Logic)

Input: New proposal from task T_i to Fog Node F_j, current set of tentatively accepted tasks at F_j, priority rankings $\succ$, and capacity quotas (q_c, pr_c, qr_c)
Result: Updated set of tentatively accepted tasks at F_j

Action: F_j tentatively accepts T_i.
Constraint Checks:
while F_j violates any constraint **do**
  if Accepted Major tasks $> qr_c$ **then**
    Reject the lowest-ranked **Major** task currently held by F_j.
  end if
  if Accepted Minor tasks $> q_c - pr_c$ **then**
    Reject the lowest-ranked **Minor** task currently held by F_j.
  end if
  if Total accepted tasks $> q_c$ **then**
    Find lowest-ranked Major task (M_{worst}) and Minor task (m_{worst}).
    Reject whichever is ranked lower overall between M_{worst} and m_{worst}.
  end if
end while

The outer loop of the matching engine handles the multi-stage quota enforcement, mathematically defined in Algorithm 2. To formally track the assignment process across multiple iterations, the following notations are introduced:

- k: The current iteration index of the multi-stage algorithm.
- V^k: The set of active tasks making proposals during iteration k.
- vs^k, vr^k, vt^k: The aggregated target minimum quotas remaining across all fog nodes for Minor tasks, Major tasks, and all tasks combined, respectively, at iteration k.
- μ^k: The tentative matching outcome resulting from the Modified DA execution in iteration k.
- $p_j^k, q_j^k, pr_j^k, qr_j^k$: The dynamically updated remaining quotas for fog node F_j at iteration k.

By iteratively tightening the active set of tasks V^k based on the remaining aggregate quotas, the engine guarantees that Major tasks secure their minimum quotas pr_j without

exceeding their strict ceilings qr_j. Once targets are met, the algorithm recursively branches to execute standard DA or MSDA for the remaining subgroups.

4.5 Algorithmic Complexity

A single Deferred Acceptance round has a complexity of $O(T \times F)$ where T is the number of tasks and F is the number of fog nodes. The MSDA stages are inherently bounded by T because each stage successfully matches at least one task or exhausts a quota. Therefore, the worst-case overall complexity is $O(T^2 \times F)$. In practice, convergence is significantly faster because the tight quota constraints limit the number of required matching stages. The Modified DA adds a negligible constant-factor overhead for enforcing the three simultaneous constraints.

Algorithm 2 Adapted 2-Type MSDA Algorithm

Input: Set of tasks $T = T_{Major} \cup T_{Minor}$, Set of Fog Nodes F, priority rankings/preferences $\succ$, and initial quotas for each fog node (p_c, q_c, pr_c, qr_c)

Result: Final stable matching outcome μ between tasks and fog nodes

Set $k = 0$, $V^0 = T$, $p_j^1 = p_c$, $q_j^1 = q_c$, $pr_j^1 = pr_c$, and $qr_j^1 = qr_c$ for all $F_j \in F$.
repeat
  $k = k + 1$
  Let $vs^k = \sum_{F_j \in F} \max(0, p_j^k - qr_j^k)$, $vr^k = \sum_{F_j \in F} pr_j^k$, and $vt^k = \sum_{F_j \in F} p_j^k$.
  Set V^k be the minimum set of tasks with the lowest priority according to $\succ$ which satisfies $|V^k| \geq vt^k$, $|V^k \cap T_{Minor}| \geq vs^k$, and $|V^k \cap T_{Major}| \geq vr^k$.
  if $V^{k-1} \setminus V^k \neq \emptyset$ **then**
    Execute Modified DA (Algorithm 1) on the tasks in $V^{k-1} \setminus V^k$. Let μ^k be the matching in this round.
    for each $F_j \in F$ **do**
      $q_j^{k+1} = q_j^k - |\mu^k(F_j)|$,
      $qr_j^{k+1} = \min(qr_j^k - |\mu^k(F_j) \cap T_{Major}|, q_j^{k+1})$,
      $pr_j^{k+1} = \max(0, pr_j^k - |\mu^k(F_j) \cap T_{Major}|)$, and
      $p_j^{k+1} = \max(0, p_j^k - |\mu^k(F_j) \cap T_{Minor}| - \max(pr_j^k, |\mu^k(F_j) \cap T_{Major}|)) + pr_j^{k+1}$.
    end for
  else
    Let $F'(\subseteq F)$ be the set of fog nodes which satisfies $p_j^k > 0$.
    if $|V^k \cap T_{Major}| == vr^k$ **then**
      Execute Standard DA on $V^k \cap T_{Major}$ and every fog node $F_j \in F'$ with $pr_j = qr_j = qr_j^k$. (For other $F_{j'} \notin F'$, set $pr_{j'} = qr_{j'} = 0$.)
      Execute Single-Type MSDA on $V^k \cap T_{Minor}$ with $p_j = p_j^k - pr_j^k$ and $q_j = q_j^k - pr_j^k$ for $F_j \in F'$ and $p_{j'} = 0$, $q_{j'} = q_{j'}^k$ for $F_{j'} \notin F'$.
      exit /* end of the algorithm */
    else if $|V^k \cap T_{Minor}| == vs^k$ **then**
      Execute Standard DA on $V^k \cap T_{Minor}$ and every fog node $F_j \in F'$ with $p_j = q_j = \max(p_j^k - qr_j^k, 0)$. (For other $F_{j'} \notin F'$, set $p_{j'} = q_{j'} = 0$.)
      Execute Single-Type MSDA on $V^k \cap T_{Major}$ with $pr_j = pr_j^k$ and $qr_j = qr_j^k$ for $F_j \in F'$ and $pr_{j'} = 0$, $qr_{j'} = qr_{j'}^k$ for $F_{j'} \notin F'$.
      exit /* end of the algorithm */
    else
      /* $|V^k \cap T_{Major}| > vr^k$ and $|V^k| == vt^k$ */
      Execute Adapted 2-Type MSDA on tasks in V^k and every fog node $F_j \in F'$ with $pr_j = p_j = pr_j^k$, $qr_j = \min(qr_j^k, p_j^k)$, and $q_j = p_j^k$. (For other $F_{j'} \notin F'$, set $p_{j'} = q_{j'} = pr_{j'} = qr_{j'} = 0$.)
      exit /* end of the algorithm */
    end if
  end if
until forever

5 Implementation and Experimental Setup

This chapter describes the Java implementation architecture and the experimental setup used to evaluate the proposed algorithm.

5.1 Java Architecture

The entire system was implemented in Java (JDK 8+) using a modular pipeline architecture across 14 classes and approximately 2,500 lines of code. Each phase of computation is a separate, self-contained class, and data flows through the pipeline via a central data transfer object (SimulationData). Figure 5.1 shows the overall architecture.

Category	**File**	**Responsibility**
Orchestrator	Main.java	Entry point; runs full pipeline across scenarios
Data Models	Task.java	Task attributes and per-fog metric arrays
	FogNetwork.java	FN attributes, capacity, and quota fields
	SimulationData.java	DTO bundling all computed arrays
	SimulationConfig.java	Central constants and parameter ranges
Generators	TaskGenerator.java	Random IoT task generation
	FogNetworkGenerator.java	Random fog node generation
Calculators	OffloadingCalculator.java	Delay and energy computation
	Normalizer.java	Min-Max + severity-weight normalisation
	UrgencyCalculator.java	Multi-criteria urgency scoring
	QuotaDeterminator.java	Min/max quota computation
Ranking & Matching	WeightGenerator.java	AHP 4×4 with CR ≤ 0.1
	PreferenceRanker.java	Fog-side rankings and precedence lists
Matching	MSDAlgorithm.java	Single-Type MSDA, 2-Type MSDA, GPM
Output	SimulationPrinter.java	Results, tables, and utilisation bars
	SimulationMetrics.java	Accumulator for averaging across iterations

Table 5.1: File structure and responsibilities of the 14-class Java implementation.

A critical design decision was the use of deep copy to ensure data independence between algorithm pipelines. All three algorithms (GPM, Baseline, Proposed) start from identical

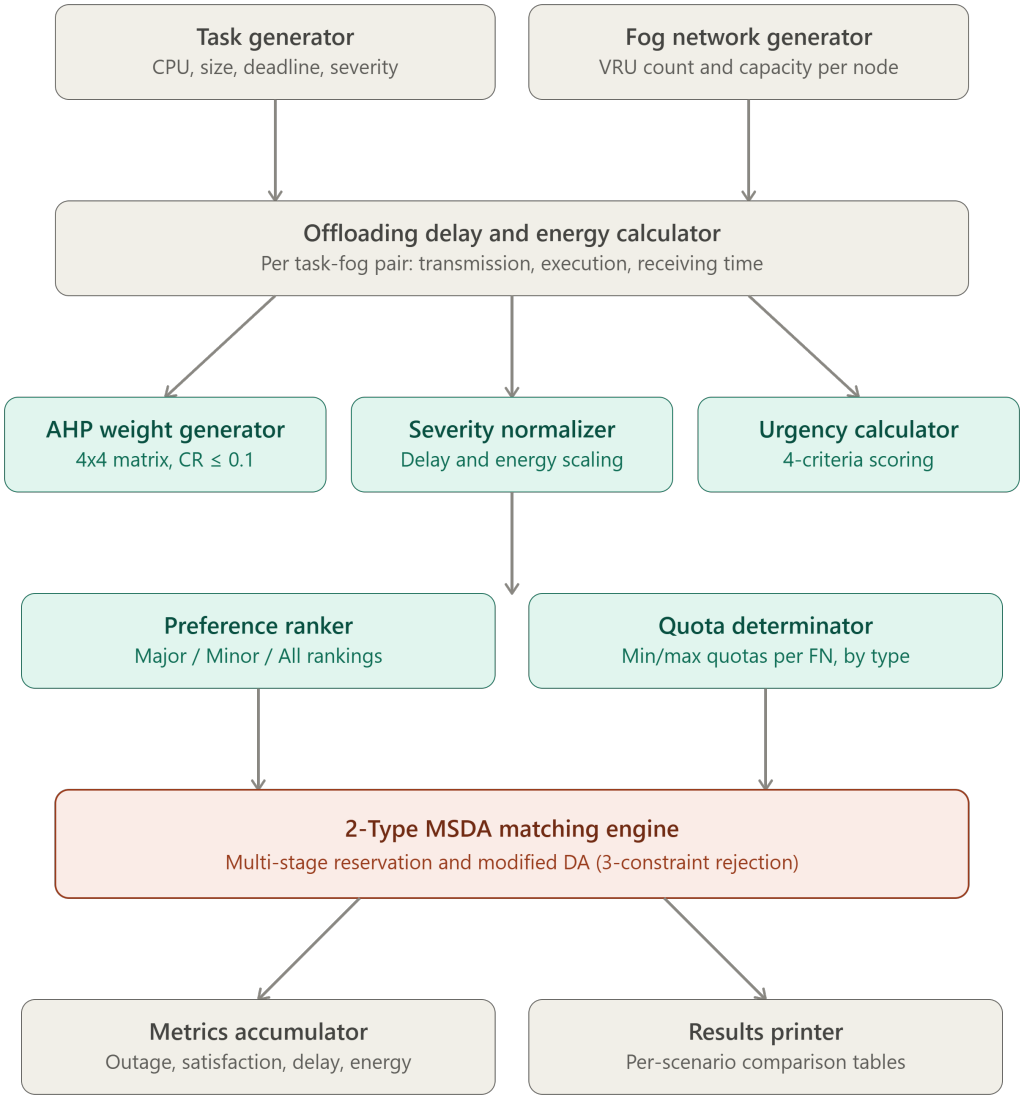

Figure 5.1: Modular pipeline architecture of the Java implementation showing the 14 classes and data flow.

raw data (same fog nodes, same tasks, same delays, same energies). The deep copy creates independent copies so each algorithm modifies its own data without contaminating the others, guaranteeing a fair apples-to-apples comparison.

5.2 Experimental Setup

5.2.1 Algorithms Compared

The three algorithms are evaluated and compared:

(i) **Greedy Placement Method (GPM)**: A baseline assignment strategy that allocates task to fog nodes based purely on available capacities. In this approach, fog nodes were sorted by their remaining computing resources, and tasks has assigned sequentially to the node with the highest available capacity that can satisfy the task's requirements. GPM do not consider task severity, multi-criteria urgency, phase (e.g., pre-surgery vs. post-surgery), or mutual preferences. As a result, while computationally inexpensive, it serves as a lower bound comparison because its lack of fairness mechanisms typically leads to the starvation of highly critical tasks and imbalances resource utilization during periods of high network load.

(ii) **Baseline (M-DAFTO)**: Single-Type MSDA with 2-criteria urgency, 2×2 AHP, and delay-only normalisation [1].

(iii) **Proposed System (Adapted 2 Type MSDA)**: severity aware 2-Type MSDA with 4-criteria urgency, 4×4 AHP, severity-weighted normalization, and separate major/minor quota tracking.

5.2.2 Simulation Parameters

The experimental setup follows standard fog computing evaluation practice [10]. Four task scales (250, 500, 1000, 2000) represent increasing load levels from normal operation to extreme overload. Each scenario uses 5 randomly generated fog nodes. Results are

averaged over 10 independent iterations to smooth out randomness, giving a total of $4 \times 10 \times 3 = 120$ algorithm runs.

Parameter	Range / Value
Number of tasks	250, 500, 1000, 2000 (per scenario)
Number of fog nodes	5
CPU demand	[210.0, 480.0] Mcycles
Input size	[300.0, 600.0] KB
Output size	[10.0, 20.0] KB
Deadline	[15.0, 22.0] seconds
Fog CPU capacity	[6.0, 10.0] GHz
Fog VRU count	[200, 500]
Phase	"Pre-Surgery" / "Post-Surgery"
Severity	"Minor" / "Major" (50/50)
Uplink data rate	2500 KB/s (20 MHz)
Downlink data rate	2500 KB/s (20 MHz)
Transmission power	0.5 W
Execution power	1.0 W
Receiving power	0.5 W

Table 5.2: Simulation parameters and their ranges, adapted from [1].

5.2.3 Metrics

The following metrics were measured for each algorithm run:

- Overall outage probability (%)
- Major task outage probability (%)
- Minor task outage probability (%)
- Total and average offloading delay (seconds)
- Total energy consumed (Joules)
- Task satisfaction (%) — measures how close each task's assignment is to its most preferred fog node
- FN satisfaction (%) — measures how well each fog node's assigned tasks match its preference ranking

6 Evaluation and Results

This chapter presents the experimental results across four task scales, comparing all three algorithms.

6.1 Overall and Severity-Specific Outage Analysis

Table 6.1 presents the overall outage probability averaged over 10 iterations per scenario.

Scale	Greedy Placement	Baseline (M-DAFTO)	Proposed System
250	0.00%	0.92%	**0.16%**
500	0.00%	1.34%	**0.02%**
1000	0.00%	3.65%	**0.02%**
2000	13.15%	20.46%	**13.15%**

Table 6.1: Overall outage probability (%) across four task scales.

Under normal-to-high load (250–1000 tasks), the proposed method reduces overall outage dramatically: from 0.92% to 0.16% at 250 tasks, from 1.34% to 0.02% at 500 tasks, and from 3.65% to 0.02% at 1000 tasks. Under extreme overload (2000 tasks), the proposed method matches GPM at 13.15%, significantly better than the baseline's 20.46%.

Table 6.2 shows the Major task (critical operations) outage — the most important safety metric.

Scale	Greedy Placement	Baseline (M-DAFTO)	Proposed System
250	0.00%	1.06%	**0.00%**
500	0.00%	1.32%	**0.00%**
1000	0.00%	3.72%	**0.00%**
2000	13.02%	20.46%	**9.36%**

Table 6.2: Major task outage probability (%) across four task scales.

The most important result is that under normal load (250–1000 tasks), the proposed

algorithm achieves **0% Major outage** — zero critical surgeries fail to get assigned. The baseline has 1–4% Major outage at the same scales. Under extreme overload (2000 tasks), Major outage drops from 20.46% to 9.36%, a 54.3% reduction even when the system is saturated.

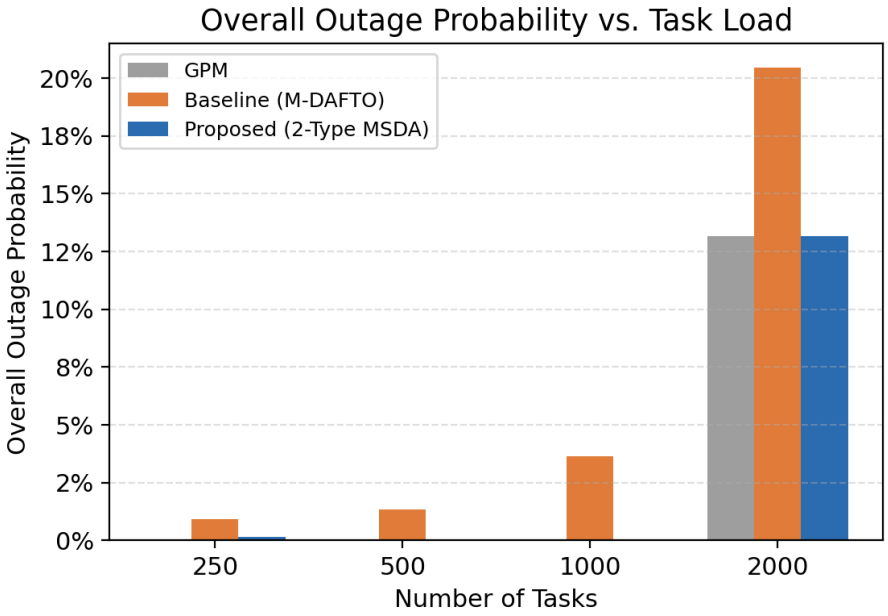

Figure 6.1: Overall outage probability comparison across four task scales.

Table 6.3 summarises the outage reduction achieved by the proposed method relative to the baseline.

Scale	Overall Outage Reduction	Major Outage Reduction
250	82.6%	∼100%
500	98.5%	∼100%
1000	99.5%	∼100%
2000	35.7%	54.3%
Average	**∼79%**	**∼89%**

Table 6.3: Outage reduction of the proposed method relative to the baseline M-DAFTO.

The S-4 scenario (2000 tasks) represents extreme overload where total task demand exceeds the combined capacity of all fog nodes. Even in this saturated case, the proposed method still reduces Major outage by 54.3%. The reduction is smaller than at lower scales because there are simply not enough fog node slots for all tasks, regardless of the

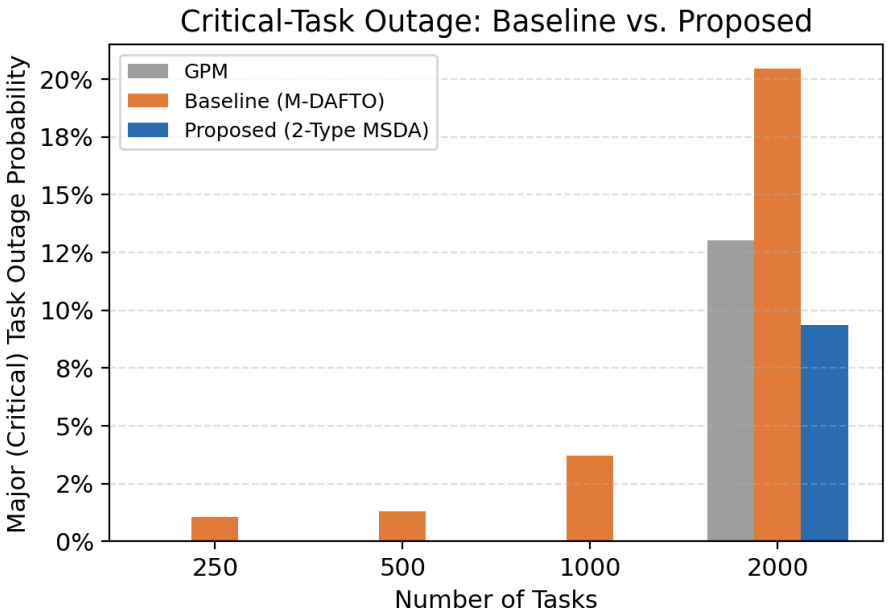

Figure 6.2: Major task outage probability comparison across four task scales.

matching algorithm used.

6.2 Delay, Energy, and Satisfaction Trade-offs

6.2.1 Delay and Energy Comparison

Scale	Greedy Placement		Baseline		Proposed	
	AvgDelay	Energy	AvgDelay	Energy	AvgDelay	Energy
250	20.66	5142.81	12.65	3102.92	12.87	3189.85
500	20.43	10168.93	12.67	6186.92	12.83	6366.47
1000	18.07	17976.69	13.36	12739.14	13.70	13604.51
2000	16.17	28033.18	14.53	22967.31	16.22	28110.36

Table 6.4: Average delay (seconds) and total energy (Joules) across all algorithms.

The proposed method has slightly higher average delay than the baseline (e.g., 12.87 vs 12.65 at 250 tasks). This is because the 2-Type MSDA sacrifices delay-optimal placement to ensure Major tasks are assigned to fog nodes that satisfy the severity-aware quota constraints. In healthcare, this is an acceptable trade-off: a minor increase in processing delay is far preferable to a critical surgery going unassigned.

6.2.2 Task and FN Satisfaction

Scale	Greedy Placement	Baseline (M-DAFTO)	Proposed System
250	9.04%	71.14%	69.96%
500	14.09%	68.26%	67.46%
1000	21.72%	60.62%	59.31%
2000	33.77%	40.27%	32.83%

Table 6.5: Task satisfaction (%) across four task scales.

Task satisfaction is slightly lower for the proposed method (e.g., 69.96% vs 71.14% at 250 tasks) because the severity-aware matching sometimes assigns tasks to their second or third preferred fog node to ensure Major quotas are met. However, the difference is small (within 2 percentage points at most scales).

Scale	Greedy Placement	Baseline (M-DAFTO)	Proposed System
250	10.04%	10.54%	10.34%
500	12.05%	19.50%	18.85%
1000	25.00%	34.57%	35.25%
2000	50.43%	50.22%	**59.08%**

Table 6.6: FN satisfaction (%) across four task scales.

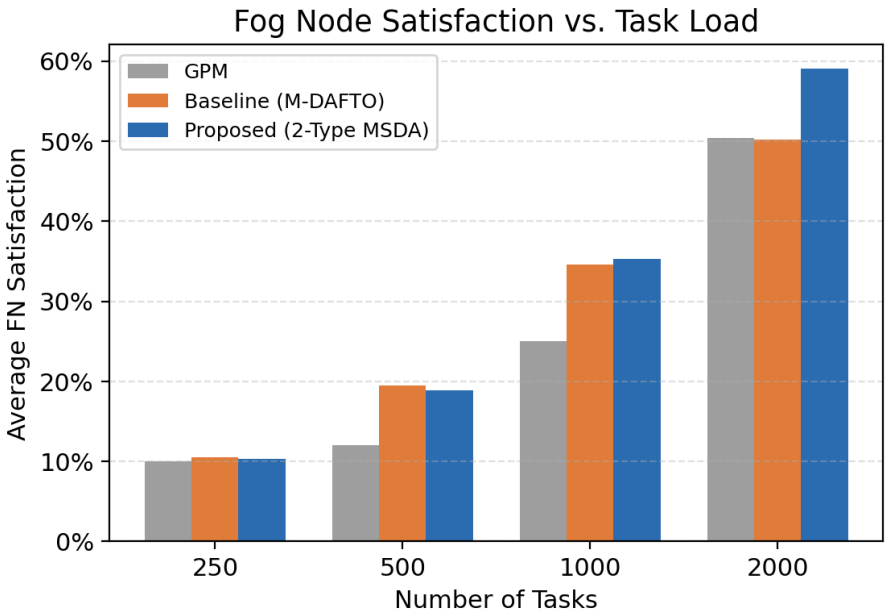

Figure 6.3: FN satisfaction comparison across four task scales.

FN satisfaction improves under overload (59.08% vs 50.22% at 2000 tasks), showing that the 2-Type system distributes tasks more effectively by leveraging separate Major/Minor quota tracking.

6.2.3 GPM Comparison

GPM shows 0% outage at 250–1000 tasks because it simply fills fog nodes greedily by capacity. However, its task satisfaction is extremely low (9–34%) because it ignores preferences entirely. GPM fails at 2000 tasks (13.15% outage) because it has no quota-based fairness mechanism. This confirms that greedy approaches, while simple, are unsuitable for healthcare IoT where both task satisfaction and critical-task protection matter.

7 Conclusion

7.1 Summary of Contributions

This project addressed the limitations of the M-DAFTO algorithm for healthcare IoT-fog task offloading by adapting the 2-Type Multi-Stage Deferred Acceptance (MSDA) algorithm from Oomori and Manabe [5] and combining it with several original extensions.

Adaptations of Prior Work

(i) **2-Type MSDA matching**: Adapted the 2-Type MSDA algorithm from [5] for IoT-fog task offloading, including the Modified DA with three simultaneous constraints and the final-stage three-case logic.

(ii) **Two-type task classification**: Applied the R-type/S-type framework from [5] by mapping Major/Minor healthcare task severity onto it, with separate quota constraints (pr_c, qr_c) per fog node ensuring critical tasks are never starved of resources.

(iii) **Separate preference lists**: Implemented separate fog-side rankings for all tasks, Major tasks only, and Minor tasks only, as structurally required by the 2-Type MSDA architecture [5].

Original Technical Additions

(i) **Energy as a metric**: Extended the urgency function to include energy consumption, computed from transmission, execution, and receiving power.

(ii) **Severity score**: Introduced a severity multiplier γ (1.5 for Major, 1.0 for Minor) to the urgency formula, naturally prioritising critical tasks.

(iii) **Severity-weighted normalisation**: Designed a Min-Max normalisation applied to both delay and energy with severity-based weighting (0.5 for Minor, 1.0 for Major).

(iv) **4×4 AHP**: Extended the AHP matrix from 2×2 to 4×4 with genuine consistency validation (CR $\leq$ 0.1).

(v) **GPM baseline**: Implemented a Greedy Placement Method as an additional comparison point.

On average across all tested scales, the adapted and extended method reduced overall outage by approximately 79% and critical-task outage by approximately 89% compared to the baseline. Under normal load (250–1000 tasks), the proposed algorithm achieves 0% Major outage. These improvements come with acceptable trade-offs in delay and task satisfaction.

7.2 Limitations

Several limitations should be noted are:

- **Batch processing**: The current system processes all tasks simultaneously in batch. The real hospitals have continuous task arrivals that would require online/streaming processing.
- **Synthetic data**: The evaluation uses randomly generated data rather than real healthcare workload traces. Testing with actual hospital data would validate practical viability.
- **Narrow deadline range**: The deadline range of 15–22 seconds may not capture the full spectrum of health care urgency. Tasks requiring sub-second processing were not tested.
- **Device-to-fog only**: The model support only device-to-fog offloading. Fog-to-fog horizontal and fog-to-cloud vertical offloading were not considered.

7.3 Future Work

Several directions warrant future investigation:

- **Dynamic task arrival**: Extending the system to handle continuous, online task arrivals rather than batch processing, making it more realistic for hospital environments.
- **Task migration**: Allowing tasks to be migrated between fog nodes when loads change dynamically, potentially improving performance under fluctuating conditions.
- **Real-world workload traces**: Testing with actual healthcare workload traces from hospitals to validate the algorithm's practical viability.
- **Heterogeneous deadlines**: Expanding the deadline range to include truly time-critical tasks (sub-second) and longer-horizon tasks, stressing the algorithm further.
- **Fog-to-fog and fog-to-cloud offloading**: Adding horizontal (fog to fog) and vertical (fog to cloud) offloading to handles overflow scenario more gracefully.
- **Multi-tier severity**: Extending from 2 severity levels to 3 or more (e.g., Critical/Major/Minor/Routine) for finer-grained control.
- **Machine learning weight generation**: Replacing random AHP weights generation with learned weights by historical data, potentially improving performance.

Bibliography

[1] C. Swain, M. N. Sahoo, and A. Satpathy, "M-DAFTO: Multi-stage deferred acceptance based fair task offloading in IoT-fog systems," *IEEE Transactions on Services Computing*, vol. 17, no. 3, pp. 1057–1070, 2024.

[2] F. Chiti, R. Fantacci, and B. Picano, "A matching theory framework for tasks offloading in fog computing for IoT systems," *IEEE Internet of Things Journal*, vol. 5, no. 6, pp. 5089–5096, 2018.

[3] D. Gale and L. S. Shapley, "College admissions and the stability of marriage," *The American Mathematical Monthly*, vol. 69, no. 1, pp. 9–15, 1962.

[4] D. Fragiadakis, A. Iwasaki, P. Troyan, S. Ueda, and M. Yokoo, "Strategyproof matching with minimum quotas," *ACM Transactions on Economics and Computation*, vol. 4, no. 1, pp. 1–40, 2016.

[5] R. Oomori and Y. Manabe, "Strategyproof matching with maximum and minimum quotas for two types of members," in *Proceedings of the 25th International Conference on Principles and Practice of Multi-Agent Systems (PRIMA 2025)*, ser. Lecture Notes in Artificial Intelligence, vol. 16366. Springer, 2026, pp. 1–16.

[6] A. E. Roth and M. A. O. Sotomayor, *Two-Sided Matching: A Study in Game-Theoretic Modeling and Analysis*, ser. Econometric Society Monographs. Cambridge University Press, 1990.

[7] Y. Kamada and F. Kojima, "Efficient matching under distributional constraints: Theory and applications," *American Economic Review*, vol. 105, no. 1, pp. 67–99, 2015.

[8] C. Swain, M. N. Sahoo, and A. Satpathy, "LETO: An efficient load balanced strategy for task offloading in IoT-fog systems," in *Proceedings of the IEEE International Conference on Web Services (ICWS)*, 2021, pp. 564–573.

[9] T. L. Saaty, *The Analytic Hierarchy Process*. New York: McGraw-Hill, 1980.

[10] H. Gupta, A. V. Dastjerdi, S. K. Ghosh, and R. Buyya, "iFogSim: A toolkit for modeling and simulation of resource management techniques in the internet of things, edge and fog computing environments," *Software: Practice and Experience*, vol. 47, no. 9, pp. 1275–1296, 2017.